Future Generation Computer Systems 00 (2027) 1–9

Who should look at this image? Small language agents choosing the tools of inspection drones

Mahdi Khashan∗

[a][Department, Institution], [City], [Country]

Abstract

Inspection drones can run small vision models on board and send images to larger models in the cloud. For each image, the drone must choose which models to use, with limited time and a weak radio link. We test whether a small language model can make this choice for power-line inspection. Step by step, the model picks one of about 60 vision tools and reads its answer. At the end it writes a report with three parts: the component type, whether there is a defect, and the condition. We train this agent by supervised fine-tuning and then by reinforcement learning, and compare its reports with the true labels. First, the set of tools matters most. With one large general model in the tool set, a small agent learns to call it and stop, and 88.1% of its reports are fully correct. When we replace that model with a small component classifier, the same training gives an agent that uses two small classifiers on the drone: 95.9% of its reports are fully correct, in 0.09 seconds per image, with no upload. Second, reasoning text from a larger teacher model, larger agents and reinforcement learning do not improve the accuracy when the tool set stays the same. Third, reinforcement learning helps when the tool set changes. When we remove the general model at test time, an agent trained with reinforcement learning uses a tool that it never saw in training: 92.4% of its reports are fully correct, against 76.2% without reinforcement learning.

Keywords: drone inspection, edge intelligence, tool selection, small language models, reinforcement learning

1. Introduction

Drones are changing how power lines, bridges and towers are inspected. A drone takes many detailed images during a flight, and new drones can also analyse these images on board [1]. For this, a drone can use many AI models. We call them *tools*. Some are small classifiers on the drone. Some are vision-language models (VLMs) on the drone. Others are large models in the cloud, and to use them the drone must upload the image over a radio link that is often slow or down. Farahani et al. run a fine-tuned SmolVLM-256M on a Jetson board and need 1.4–6.4 s for each video segment [2]. Azimi et al. compare three ways to run such a model: on the drone, in the cloud, or split between the two [3]. Both groups say that an adaptive controller, which decides during the flight which tool to use, is future work.

Language models that use tools are now common [8, 9, 10]. Zhou et al. [5] show that even small language models can learn to select tools from a long list. They train the model first with supervised fine-tuning (SFT) and then with a reinforcement learning method called GRPO [6]. A small model of this kind could run next to the other software of the drone and plan the inspection of each image.

This article asks two questions: *can a small language model learn to choose the tools of an inspection drone, and what does reinforcement learning add?* We build a set of about 60 vision tools and save their answers for every image. Then we let Qwen3 language models [7] with 0.6, 1.7 and 4 billion parameters work as agents: they call tools step by step and then write a report. We train them with the method of Zhou et al., with and without reasoning text written by a larger teacher model. We compare them with the same models without training and with each tool alone. We also remove tools at test time to see if the agents can adapt.

Our main contributions are:

- a tool-selection benchmark for drone inspection with 62 real vision tools, measured run times, recorded mobile-network data and an upload cost;
- evidence that the tool set decides the accuracy: when one large general model is replaced by a small specialist, fully correct reports rise from 88.1% to 95.9%, and no upload is needed;
- evidence that teacher reasoning, larger models and GRPO do not improve the accuracy when the tool set stays the same, and an explanation of why;
- evidence that GRPO helps agents adapt to a changed tool set: without new training, a 1.7B agent uses a tool it never saw in training and reaches 92.4%, against 76.2% without GRPO and 95.9% after new training.

Section 2 reviews related work. Section 3 describes the data, the tools, the agent and the training. Section 4 presents and discusses the results, and Section 5 concludes.

∗Corresponding author.
Email address: mahdikhashan1@gmail.com (Mahdi Khashan)

2. Related Work

Edge VLMs for inspection. EVLM [2] runs a SmolVLM-256M with LoRA on a Jetson and reduces the upload of a 4 s video segment from 485 kB to 25 kB. The study in [3] measures SmolVLM on the drone, in the cloud, and split between them, for different link speeds. INAR-VL [4] sends each request to the edge or to the cloud, depending on the input and the link speed. Farahani et al. [1] show that SmolVLM-256 needs about 2.8 s per image on a Jetson Orin, while a YOLO detector needs 0.14 s. They conclude that a light model should start the VLM only when it is needed. All these systems use fixed rules to decide where one model runs. They do not choose, for each image, among many models with different strengths, costs and locations.

Language models that use tools. Toolformer [8] and ReAct [9] let a language model call tools inside its own text. ToolLLM [10] and Gorilla [11] train models on thousands of software interfaces and use a search step to shorten the list of tools. Zhou et al. [5] split the problem into planning, search and selection. They show that small models trained with SFT and GRPO can also use new tools. Other work shows that a small model becomes better when it learns from the written reasoning of a larger model [12]. These studies use software tools with no time limit, no network and no real cost per call. Their tools rarely disagree, and success usually means a correct call, not a correct final answer.

Our earlier work. On the same data, we trained a small policy with PPO [15] to choose among four tools. It was as good as well-tuned rules but not better, and a fixed formula that combined the tool answers gave the largest gain. In this article the language model itself reads the tool answers and writes the report, so we use no such formula.

3. Methodology

The method has four parts. We first describe the data and the tools (Sections 3.1–3.2) and what a tool call costs (Section 3.3). Then we define the task of the agent, the report, and how the agent works step by step (Sections 3.4–3.5). Finally we define how the agent learns: the training targets, SFT and GRPO (Section 3.6), and the variant with teacher reasoning (Section 3.7).

3.1. Data

We use the component images of InsPLAD [23] with the `tower14` split of ElecVQA-Bench [22]. The 18 training towers are used to train the tools. The 10 validation towers (5,624 images) are used to build the training data of the agent. The 14 test towers (6,092 images) are used for testing. There are 11 component types and six conditions: good, rust, corrosion, missing cap, bird nest and other defect. Only 9.9% of the test images show a defect.

3.2. Tools

Table 1 shows the 62 tools in groups. We fine-tuned the classifiers on the training towers. We ran every tool once on every validation and test image and saved the answers. During training and testing, the agent gets these saved answers. The three laya-vision tools [18] answer all three questions: component, defect and condition. We added a ResNet-18 component classifier later (Section 4.3); the main experiment uses the other 61 tools. Fourteen tools run on the drone, for example ResNet-18, DeiT-S, MobileNet, the component classifier, a compressed laya-vision and SmolVLM. The other 48 tools run in the cloud.

Table 1. The 62 tools in groups. Answer: the report fields that the tool can answer (C: component, D: defect, S: condition).

Group	Examples	Tools	Answer
Image classifiers	ResNet-18/50/101 [19], ConvNeXt, ViT, DeiT, Swin, EfficientNet, MobileNet, YOLO	12	D, S
Classifier variants	mirror image, five crops, new cut from the full photo	6	D, S
Component specialists	ResNet-50 and ConvNeXt for each component group	10	D, S
Feature classifiers	DINOv2 [20], CLIP [21]: condition classifier and defect detectors	15	D or S
Zero-shot	CLIP-L with text prompts	1	D, S
Small VLMs	SmolVLM-256M [17], 3 variants	3	D
laya-vision	full, compressed (4-bit), new cut	3	C, D, S
Large VLM	Qwen3-VL-8B, 8 questions or cuts	8	D
Component classifier	ResNet-18 on 11 component types	1	C
Image checks	quality, exposure, size	3	–

3.3. Time, network and limits

Each tool call costs time. We measured the run time of each tool on a desktop GPU. We have no Jetson board, so we multiply the time of a drone tool by 9. This is our estimate of how much slower a Jetson Orin NX is.

A cloud tool needs its run time, a network round trip, and the time to upload the image. Let g_t be the measured run time of tool t, b the size of the image in bits, v the link speed in bits per second, and $u = 1$ if the image is not uploaded yet ($u = 0$ otherwise). The time of a call is

$$\tau(t) = \begin{cases} 9\, g_t & t \text{ runs on the drone} \\ g_t + 0.05\,\text{s} + u\,\dfrac{b}{v} & t \text{ runs in the cloud.} \end{cases} \quad (1)$$

The drone uploads each image only once, so later cloud tools for the same image do not pay the upload again. If the link is down ($v = 0$), no cloud tool can be called. The link speed follows recorded 4G/LTE data [24], reduced to 10% (about 3 Mbit/s). A typical image has 33 kB.

The agent has a time budget B of 4, 10 or 30 s for each image. Let K be the set of tools already called and T the time used so far. A tool can be called only if it was not called before and its

time fits in the time that is left:

$$F = \{\, t \notin K : \; T + \tau(t) \le B \,\}. \tag{2}$$

The agent has at most 8 steps; at step 8 it must write the report.

3.4. The report

For each image, the agent must answer the same three questions that laya-vision answers:

1. *Which power line component is in this image?* One of 11 types, for example glass insulator, shackle or yoke suspension.
2. *Does this power line component have a defect?* Yes or no.
3. *Which condition does this power line component have?* One of six conditions: good, rust, corrosion, missing cap, bird nest or other defect.

The agent gives the answers as one short JSON object, for example

```
{"name": "REPORT", "component": "glass insulator",
   "defect": "yes", "condition": "missing cap"}
```

A field is correct only if it is exactly equal to the true label. Figure 1 shows four test images with their true labels and the answers of two tools. Tables 3–7 give the share of correct answers for each question.

3.5. The agent: plan, select and report

The agent is a text-only language model. It never sees the image. It only reads the answers of the tools as short text. For each image it repeats three stages until it writes the report.

Plan. The agent reads the state: the task, the time left, the state of the link, whether the image is already uploaded, and the tool answers so far. Zhou et al. [5] train a separate planning step. We do not. In the agents without thinking, the plan is implicit in the next choice. In the thinking agents, the written reason before each action plays the role of a short plan (Section 3.7).

Select. A text search ranks the tools (see below), and the agent receives the 15 best tools of the set F (Eq. (2)), plus the option *report*. It answers `{"name": "SELECT", "ids": ["resnet18"]}`, for example. The saved answer of this tool is then added to the next prompt as one line of text, such as *"resnet18: missing cap 99%, good 1%"*.

Report. When the agent thinks it knows enough, it chooses *report* and writes the three answers (Section 3.4). The work on this image then ends. At step 8 the agent must report.

Ranking of the tools. With 62 tools, a long list would fill the prompt, so we show only 15. We rank the tools with BM25 [13], a standard text-search score. Each tool is a short document: its name, its location (drone or cloud) and a one-line description. The query is a fixed text about the task ("inspect power line component defect condition component type") plus the answers of the tools called so far. A tool t gets the score

$$\text{score}(t) = \sum_{w \in \text{query}} \text{idf}(w)\, \frac{n_{w,t}\,(k_1+1)}{n_{w,t} + k_1\big(1 - b + b\,\frac{L_t}{\bar{L}}\big)} \tag{3}$$

where $n_{w,t}$ is how often word w appears in the document of tool t, L_t is the length of this document, $\bar{L}$ is the mean length of all documents, and $\text{idf}(w)$ is high for words that appear in few documents. We use $k_1 = 1.8$ and $b = 1.0$ and remove common English words. We keep only the tools in F and show the 15 with the highest score. Because the query contains the tool answers, the list changes during the work on an image: after "missing cap 82%", for example, the tools that check only for a missing cap move to the top.

3.6. Learning to select: targets, SFT and GRPO

Like Zhou et al. [5], we train each step separately. We first fix the notation, then define the target of a step, and then the two training stages.

Notation. An image has the true labels $y = (y_{\text{C}}, y_{\text{D}}, y_{\text{S}})$ for component, defect and condition. At a step, s is the prompt (the state), K is the set of tools already called, and F is the set of tools that can still be called (Eq. (2)). The agent is a policy $\pi_\theta(a \mid s)$ with weights θ. An action a is either a tool $t \in F$ or a report $r = (r_{\text{C}}, r_{\text{D}}, r_{\text{S}})$. A tool t gives an answer z_t for some of the three fields. The fields that the tools in K answer correctly are

$$C(K) = \{f : \; z_{t,f} = y_f \text{ for some } t \in K\}. \tag{4}$$

Target of a step. The target a^* is the action that we want the agent to take. A tool is useful if it adds correct fields that are still missing, so we define its gain as

$$g(t) = \big|C(K \cup \{t\})\big| - \big|C(K)\big|. \tag{5}$$

The target is the tool with the largest gain. If two tools have the same gain, the faster one is the target. If no tool has a gain, the target is the report with the true labels:

$$a^* = \begin{cases} \arg\max\limits_{t \in F} \big(g(t),\, -\tau(t)\big) & \text{if } \max_{t \in F} g(t) > 0 \\ \text{report } r = y & \text{otherwise.} \end{cases} \tag{6}$$

Because the report target is y and not a tool answer, the agent must learn which tool to trust when tools disagree. Note that $g(t)$ uses the true labels. The target is therefore chosen with knowledge that the agent does not have at test time; Section 4.8 discusses the effect.

Stage 1: SFT. We run the target policy on the validation images, with a random tool in 30% of the tool steps so that the agent also sees unusual states. This gives a set $\mathcal{D}$ of 61,461 pairs (s, a^*) from 22,496 runs (each validation image with 4 budgets). SFT maximises the probability of the target:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\frac{1}{|\mathcal{D}|} \sum_{(s,a^*) \in \mathcal{D}} \log \pi_\theta(a^* \mid s). \tag{7}$$

We train LoRA adapters [16] on 12,000 pairs for one pass with the TRL library [14]. We call the result π_{SFT}.

Stage 2: GRPO. GRPO [6] starts from π_{SFT}. For a state s, the agent samples a group of $G = 8$ answers $o_1, \ldots, o_G$ from $\pi_\theta(\cdot \mid s)$. Each answer gets a reward that depends on the target

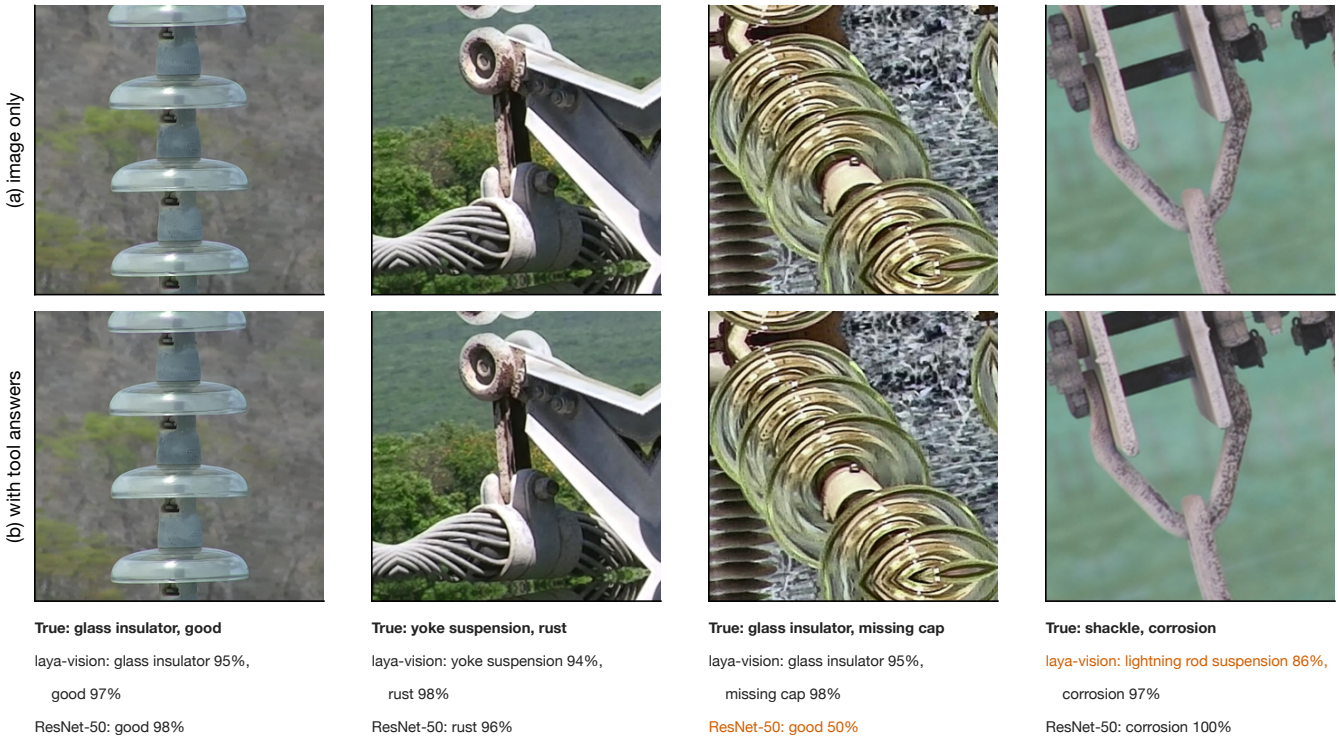


Figure 1. **Four test images, without and with tool answers.** (a) The image alone, as the drone takes it. The agent never sees it. (b) The same image with its true labels and the answers of two tools, which is all the agent reads. Orange: a wrong answer. In the third image, ResNet-50 says "good" with only 50% while laya-vision finds the missing cap. In the fourth image, laya-vision names the wrong component but the right condition.

a^* of the state:

$$R(o) = \begin{cases} \mathbb{1}[\, o = a^* \,] & \text{if } a^* \text{ is a tool} \\ \dfrac{1}{3} \displaystyle\sum_f \mathbb{1}[\, o_f = y_f \,] & \text{if } a^* \text{ and } o \text{ are reports} \\ 0 & \text{otherwise.} \end{cases} \tag{8}$$

Here f runs over the three fields C, D and S, and o_f is the answer of report o for field f. So a tool choice earns 1 only if it is the target tool, a report earns one third for each correct field, and a report that comes too early or an answer that is not valid earns 0. GRPO then compares each answer with its group:

$$A_j = \frac{R(o_j) - \mu_R}{\sigma_R + \delta}, \qquad \mu_R = \frac{1}{G} \sum_{j=1}^{G} R(o_j), \tag{9}$$

where σ_R is the standard deviation of the G rewards and δ is a very small number. The loss raises the probability of answers with $A_j > 0$, lowers it for $A_j < 0$, and keeps the policy close to π_{SFT}:

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\frac{1}{G} \sum_{j=1}^{G} \Big[\min\big(\rho_j A_j,\ \text{clip}(\rho_j, 1-\epsilon, 1+\epsilon)\, A_j\big) - \beta\, D_{\text{KL}}\big(\pi_\theta \,\|\, \pi_{\text{SFT}}\big) \Big], \tag{10}$$

with $\rho_j = \pi_\theta(o_j \mid s) / \pi_{\theta_{\text{old}}}(o_j \mid s)$, the ratio between the current policy and the policy that sampled the answers. The first term is the clipped policy-gradient term of PPO [15]. The second term is the Kullback–Leibler (KL) divergence between the current policy and the SFT policy, estimated on the tokens of the sampled answers. We use $\epsilon = 0.2$ and $\beta = 0.02$, and train for 400 steps on 4,000 states.

Two properties of Eqs. (8)–(10) are important for the results. First, if all G answers get the same reward, then $R(o_j) = \mu_R$ and every $A_j = 0$. The policy-gradient term is then zero, and the step gives *no learning signal*. This happens when the policy is so sure that all samples are the same. Second, the KL divergence $D_{\text{KL}}(\pi_\theta \,\|\, \pi_{\text{SFT}})$ measures how far GRPO has moved the policy from its SFT starting point. We report both quantities in Section 4.4.

The *base* model is the same language model with the same instructions but without training.

3.7. Teacher reasoning (thinking variant)

After SFT, the G answers in GRPO are often the same, so most steps give no learning signal. To make the answers more different, we let the agent write a short reason before each action. We asked a larger model, Claude Haiku 5.5, to write such a reason (80–150 words) for 4,000 pairs (s, a^*), with similar numbers for each target. The reason explains the state, the missing fields, the time and the link, and why a^* is a good choice. It does not mention the true labels. 3,780 reasons passed our automatic checks; Box 1 shows one of them. SFT (Eq. (7)) then uses the reason followed by a^* as its target text, and GRPO continues for 200 steps with the same reward. The reward looks

Table 2. Accuracy of single tools on all 6,092 test images. "–": the tool does not answer this field. Last row: share of images where at least one of the 61 tools of the main experiment is correct.

Tool	Runs on	Component	Defect	Condition
laya-vision	cloud	0.924	0.947	0.952
laya-vision, compressed	drone	0.870	0.944	0.938
ResNet-18	drone	–	0.981	0.979
DeiT-S	drone	–	0.982	0.980
ConvNeXt-T, new cut	cloud	–	**0.988**	**0.988**
Component ResNet-18	drone	**0.996**	–	–
At least one tool correct		0.970	1.000	1.000

only at the action, not at the reason. We trained this variant with Qwen3-0.6B, Qwen3-1.7B and Qwen3-4B, with the same data and settings. The component classifier did not exist when we trained the thinking agents.

Box 1: A teacher reason from the training data.

State: no time limit, 0.80 s used; link up, 2.6 Mbit/s; image already uploaded.
Tool answers so far:

```
efficientnet: missing cap 100%, corrosion 0%
laya: component glass insulator 88%; defect 97%;
missing cap 98%, good 1%
```

Reason (Haiku 5.5): "I have used 0.80 s with no time limit, and the image is uploaded at 2.6 Mbit/s. Laya reports glass insulator at 88%, with defect at 97% and missing cap at 98%, and good, no defect at only 1%. Efficientnet reports missing cap at 100% and corrosion at 0%. Two different models agree on missing cap, and the defect reading is high. The condition is clear, so I stop and report glass insulator with a defect and condition missing cap."

Action:

```
{"name": "REPORT", "component": "glass insulator",
"defect": "yes", "condition": "missing cap"}
```

4. Experimental Evaluation

We test the agents without thinking on 1,500 of the 6,092 test images, chosen with a fixed random seed. We test the thinking agents, and the agents with removed tools, on the first 500 of these images. We count how often each report field is correct and how often all three fields are correct. All tests use one seed, and the agent always takes its most likely answer. With 500 images, a difference of about three points can be chance.

The evaluation follows the two questions of the introduction. Sections 4.1–4.3 ask what the agent learns and how much the tool set matters. Sections 4.4 and 4.5 ask what GRPO, teacher reasoning and larger models add when the tool set stays the same. Section 4.6 asks what they add when the tool set changes.

4.1. The tools alone

Table 2 shows how accurate some tools are when each one answers alone. The fine-tuned classifiers are strong on defect and condition: a ConvNeXt on a new cut of the component is correct on 98.8% of the images. In the main experiment, only laya-vision names the component, and it is correct on 92.4% of the images. The component classifier that we added later is correct on 99.6%. If the agent always picked a correct tool among the 61 tools of the main experiment, it would reach 97.0% on the component and 100% on defect and condition.

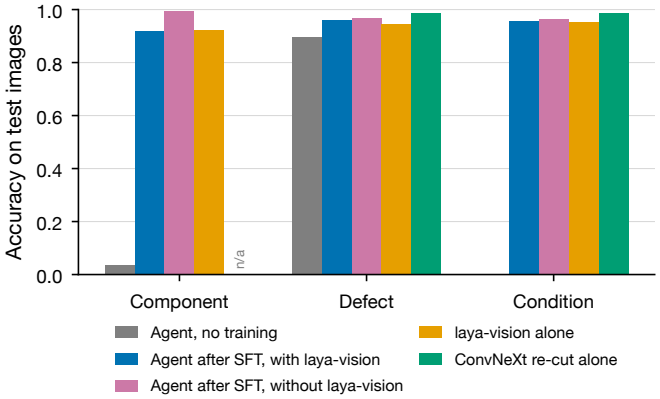


Figure 2. **The tool set decides what the agent can reach.** Accuracy for each report field at the 4 s budget. With laya-vision, the trained agent is as good as laya-vision. Without it, the agent uses two classifiers on the drone and is better on every field. Agents: 1,500 test images; single tools: all 6,092 test images; single seed. SFT and SFT + GRPO give the same results. ConvNeXt re-cut does not answer the component (n/a).

4.2. SFT teaches the answer format and one strong tool

Table 3 and Figure 2 show the accuracy of the agent's reports. Without training, Qwen3-0.6B never calls a tool. It writes a report at the first step and always says "no defect". Its defect accuracy of 0.895 is only the share of images without a defect; it finds no defect at all. Its component and condition answers are almost never correct.

After SFT, 88.1% of the reports are fully correct. The agent calls exactly one tool for each image and then writes the report. On 95% of the images the tool is laya-vision in the cloud. On the other images it is the compressed laya-vision on the drone, mostly when the link is down.

The agent is a little better than the tool it copies. Its component accuracy is the same as that of laya-vision (0.92). Its defect and condition answers are slightly better (0.961 against 0.947, and 0.958 against 0.952). The agent sees only the answer of laya-vision, so it has learned to correct some of these answers when it writes the report.

The agent does not use the best tools. A ConvNeXt on the new cut is about 3 points better on defect and condition than the agent, and DeiT-S on the drone is 2 points better. The agent never calls them. The reason is the target rule of Eq. (6). Laya-vision can add three correct fields in one call, so it has the largest gain $g(t)$ and is the target in almost every first step (24,577 of 30,950 tool steps). Another tool becomes the target only when laya-vision is wrong, and this is rare.

The budgets make no difference. One laya-vision call needs about 0.35 s. This is much less than the smallest budget of 4 s, so the agent acts in the same way for every budget.

4.3. A small specialist instead of a large general model

In the main experiment, only laya-vision names the component. Without it, the agent cannot answer all three questions. We therefore trained a ResNet-18 to name the 11 component types and added it as a drone tool. It answers only the component.

Table 3. Report accuracy of the Qwen3-0.6B agent on 1,500 test images, single seed. All three: share of reports where component, defect and condition are all correct. Time and upload: mean per image. We leave out the results without a time limit because of a fault in the time check (Section 4.8). Best results in bold.

Method	Budget	Component	Defect	Condition	All three	Defects found	Time	Upload
Base (no training)	4 s	0.037	0.895	0.000	0.000	0.000	0.00 s	0.04 kB
SFT	4 s	**0.919**	**0.961**	**0.958**	**0.881**	0.885	0.35 s	34.1 kB
SFT	10 s	0.917	**0.961**	**0.958**	0.880	**0.892**	0.37 s	34.1 kB
SFT	30 s	0.915	0.959	0.957	0.876	0.885	0.41 s	33.7 kB
SFT + GRPO	4 s	**0.919**	**0.961**	**0.958**	**0.881**	0.885	0.35 s	34.1 kB
SFT + GRPO	10 s	0.917	**0.961**	**0.958**	0.880	**0.892**	0.37 s	34.1 kB
SFT + GRPO	30 s	0.915	0.959	0.957	0.876	0.885	0.41 s	33.7 kB

It is correct on 99.6% of the test images (Table 2) and needs only 2.8 ms on a desktop GPU. Then we removed the three laya-vision tools, built the training data again with the same target rule, and trained the agent in the same way.

Table 4 shows the result. The agent now calls two tools for every image: the component classifier, and ResNet-50 (95%) or ResNet-18 (5%) for defect and condition. All of them run on the drone, so nothing is uploaded. Fully correct reports rise from 88.1% to 95.9%, and the time per image falls from 0.35 s to 0.09 s. Most of the gain comes from the component. Defect and condition improve by less than one point, and the share of defects found stays the same. GRPO again changes no answer.

The agent still does not use the best tool for defect and condition. ConvNeXt on the new cut would be about two points better. Among the tools with the same gain, Eq. (6) picks the fastest one. ResNet-50 is fast and correct on 97% of the training images, so ConvNeXt is the target in only 12 of about 84,600 training steps.

4.4. Why GRPO does not improve the accuracy

The two agents so far gave the same results with and without GRPO. Section 3.6 explains how this can happen: GRPO learns from a step only if the G answers get different rewards. Table 5 shows that after SFT this was rare. In 91–95% of the steps all 8 answers got the same reward, so every advantage A_j in Eq. (9) was zero. The SFT policy was almost fully sure of its answer (entropy about 0.002), so the 8 samples were usually the same text. As a result, the KL divergence between the GRPO policy and the SFT policy stayed near 0.002: GRPO hardly moved the policy. No most likely answer changed, and the test results are the same as after SFT in every digit.

With teacher reasoning, the 8 answers are more different. Only 62–71% of the steps gave no learning signal, and the KL divergence to the SFT policy was about 0.06, 30 times larger. But the mean reward during training did not rise (0.845 in the first half, 0.811 in the second half). So GRPO now moves the policy, but not clearly towards the targets.

4.5. Thinking and larger models

Teacher reasoning gives GRPO more learning signal. Does it also give better reports, and does a larger model help? Table 6 shows the thinking agents with all 62 tools. Teacher reasoning helps the 0.6B agent a little. At 4 s it is the best agent that has laya-vision (0.926), because it often calls the component classifier as a second tool. But its result changes between 0.856 and 0.926 for budgets that never limit it. So the budget text in the prompt changes its plan, which is not what we want.

Larger models are not better. With thinking and SFT, the 1.7B and 4B agents reach 0.876 and 0.842. After GRPO, all three sizes act in the same way as the SFT agent without thinking: they call laya-vision on 95% of the images, use one tool and reach about 0.875. Without training, the thinking models write no useful report. The 1.7B and 4B models think for too long and never give an answer.

4.6. When the tool set changes

So far the tool set at test time was the same as in training, and GRPO, reasoning and model size made little difference. In a real drone fleet, however, tools will be added, removed and replaced. So we tested what happens when the tool set changes after training. We took the agents that were trained with laya-vision and removed the three laya-vision tools at test time. We did not train the agents again. To write a correct report, an agent must now find other tools. One of them is the component classifier, which was never a target in its training data.

Figure 3 and Table 7 show the result. The SFT agents adapt badly. The 0.6B agent without thinking falls to 0.334 and never calls the component classifier. GRPO changes this. The 0.6B agent with GRPO starts from the same SFT model, but it reaches 0.754 and calls the component classifier on 72% of the images. The 1.7B agent with thinking and GRPO reaches 0.924. This is close to the agent that we trained again without laya-vision (0.959). It calls the component classifier on almost every image and then ResNet-50, like the newly trained agent.

This is surprising, because with the full tool set the GRPO agents acted exactly like their SFT models (Table 3), and the KL divergence between them was small (Table 5). A small change of the policy is enough to change its behaviour in states that it did not see in training. The effect of GRPO appears only when the tool set changes. The SFT agents often find the component classifier, but then they trust the wrong tool for the condition. The 0.6B agents also change a lot between budgets. Two tests were not finished when we wrote this draft: the 4B agent with GRPO, and the 1.7B agent with GRPO for 30 s and for no time limit (it reaches 0.910 at 10 s).

Table 4. The same training with a different tool set. Qwen3-0.6B agent at the 4 s budget, 1,500 test images, single seed. Without laya-vision, the ResNet-18 component classifier is in the tool set. Calls: tools called per image. Best results in bold.

Tools	Method	Component	Defect	Condition	All three	Defects found	Time	Upload	Calls
With laya-vision	SFT	0.919	0.961	0.958	0.881	**0.885**	0.35 s	34.1 kB	1.0
Without laya-vision	base	0.019	0.346	0.003	0.001	0.452	0.14 s	19.2 kB	0.4
Without laya-vision	SFT	**0.994**	**0.967**	**0.965**	**0.959**	0.879	**0.09 s**	**0 kB**	2.0
Without laya-vision	SFT + GRPO	**0.994**	**0.967**	**0.965**	**0.959**	0.879	**0.09 s**	**0 kB**	2.0

Table 5. GRPO training of the 0.6B agent, mean of the first and the second half of training. No signal: share of steps where all 8 answers got the same reward, so that $A_j = 0$ for all j. KL: mean $D_{\mathrm{KL}}(\pi_\theta \,\|\, \pi_{\mathrm{SFT}})$ during training.

	Without thinking	With thinking
Training steps	400	200
Answer length	about 22 tokens	about 150 tokens
No signal, 1st / 2nd half	0.91 / 0.95	0.62 / 0.71
Mean reward μ_R, 1st / 2nd half	0.861 / 0.903	0.845 / 0.811
KL to the SFT policy	≈0.002	≈0.06

Table 6. Thinking agents with all 62 tools at the 4 s budget, 500 test images, single seed. All three: fully correct reports. Calls: tools called per image. Best results in bold.

Model	Method	Comp.	Defect	Cond.	All three	Calls
Qwen3-0.6B	base	0.094	0.260	0.002	0.002	0.73
	SFT	**0.974**	0.952	0.956	**0.926**	1.89
	SFT + GRPO	0.930	0.950	0.958	0.876	1.00
Qwen3-1.7B	base	0.000	0.000	0.000	0.000	0.00
	SFT	0.918	**0.960**	**0.960**	0.876	1.25
	SFT + GRPO	0.930	0.946	0.956	0.876	1.00
Qwen3-4B	base	0.000	0.000	0.000	0.000	0.00
	SFT	0.904	0.956	0.940	0.842	1.15
	SFT + GRPO	0.930	0.946	0.952	0.874	1.01

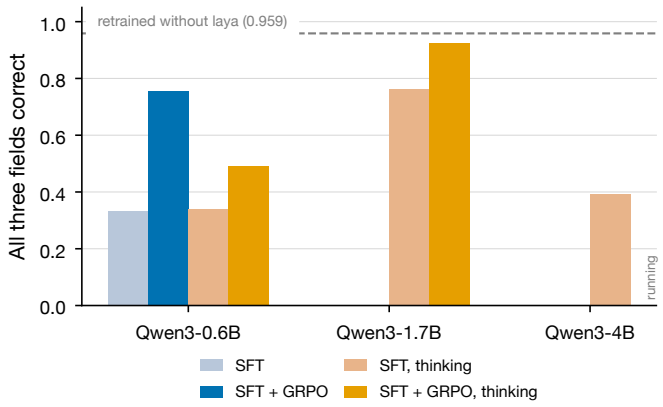


Figure 3. **GRPO helps agents adapt when the general model is removed.** Fully correct reports at the 4 s budget. The agents were trained with laya-vision; it is removed at test time, with no new training. Dashed line: agent trained again without laya-vision (1,500 images). Others: 500 test images, single seed. Agents without thinking exist only for 0.6B.

4.7. Time and upload

Table 8 compares the time and upload of our agents with published measurements on a Jetson. The numbers are not fully comparable: an EVLM segment has several video frames and a text report, while we process one image and give three answers.

Table 7. Laya-vision removed at test time, 4 s budget, 500 test images, single seed. New tool: share of images where the agent calls the component classifier, which was never a target in its training.

Model	Method	Comp.	Cond.	All three	New tool
Qwen3-0.6B	SFT	0.348	0.898	0.334	0.00
	SFT + GRPO	0.774	**0.956**	0.754	0.72
	SFT, thinking	0.718	0.482	0.340	0.73
	SFT + GRPO, thinking	0.634	0.714	0.492	0.64
Qwen3-1.7B	SFT, thinking	0.830	0.846	0.762	0.84
	SFT + GRPO, thinking	**0.988**	0.942	**0.924**	**1.00**
Qwen3-4B	SFT, thinking	0.600	0.736	0.394	0.60
Trained again without laya-vision		0.994	0.965	0.959	1.00

Table 8. Time and upload. Other work: measured on a Jetson Orin. This work: mean per test image at the 4 s budget, time of the tools only.

System	Time	Upload
Other work		
YOLO detector, per image [1]	0.14 s	–
SmolVLM-256, per image [1]	2.8 s	–
EVLM (SmolVLM), per 4 s segment [2]	1.4–6.4 s	25 kB
SmolVLM on the drone, per request [3]	2.7–2.8 s	0
This work		
SFT agent (one laya-vision call)	0.35 s	34.1 kB
SFT agent without laya-vision (two drone classifiers)	0.09 s	0
ResNet-18 on the drone, alone	0.03 s	0

Our times include only the tools, not the agent itself.

4.8. Discussion

The tool set matters more than the training method. The same training gave 7.8 more points of fully correct reports when we replaced one general model with a small specialist. GRPO changed nothing with either tool set. So it is at least as important to design and measure the tools as to train the agent.

The agent can only be as good as its targets. SFT teaches the agent the target behaviour almost perfectly. But the gain $g(t)$ in Eq. (5) uses the true labels of each training image. The target is therefore a tool that answers many fields, or a fast tool that happens to be right on this image. It is not the tool that is most accurate on average, which is what the agent needs at test time. Better targets should use the expected accuracy of a tool, and the reward should depend on the final report, not on a single step.

GRPO needs different answers, and it helps under change. After SFT with short answers, the policy is too sure, the 8 answers get the same reward, and the advantages in Eq. (9) are zero. Teacher reasoning makes the answers more different, but

it does not improve the accuracy (Table 6). Still, GRPO is what helps the agents when the tool set changes (Table 7). A possible reason is that GRPO tries other choices during training and rewards them, so the agent depends less on the exact tools it copied. Next steps are to reward every tool that is equally good, and to train on the whole inspection of an image with the final report accuracy as the reward.

Larger is not better here. With the usual tool set, the 1.7B and 4B agents are not more accurate than the 0.6B agent. Only when the tool set changes is the 1.7B agent with GRPO clearly the best. A 0.6B model is already a heavy load for a drone. So the smallest model is the best choice, unless the tool set changes often.

The budgets must be tighter. With budgets of 4 s or more and a first call of 0.35 s, time never limits the agent. We need shorter budgets, a cost for the upload, or slower drone tools before the agent must choose between accuracy and time.

Limitations. The agent works with saved tool answers. The network and the times on the drone are simulated, and the factor of 9 is an estimate. The time of the agent itself is not counted. A fault in the time check allowed cloud tools when the link was down and there was no time limit. It affects 1.1% of the training steps and all results without a time limit, so we leave these results out. The thinking agents and the tests with removed tools use 500 images, and some of these tests were not finished. All results come from one seed and one data set of power-line components.

5. Conclusion

A small language model can learn to inspect power-line images by calling tools, and the tool set decides most of its accuracy. With a large general model in the tool set, a Qwen3-0.6B agent learns to call this model and stop, and 88% of its reports are fully correct. With a small component classifier instead, the same training gives an agent that uses two classifiers on the drone. It reaches 96% in 0.09 s with no upload. Teacher reasoning, larger models and GRPO do not improve the accuracy when the tool set stays the same. The reason is that the model is too sure after SFT, and that the targets prefer cheap tools. But GRPO helps when the tool set changes. Without new training, a 1.7B agent with reasoning and GRPO uses a tool that was never a target in its training and reaches 92%, close to a newly trained agent. The tools of a drone fleet will change over time. So it is useful to train small agents with reinforcement learning, even when this gives no gain at first. In future work we will reward the final report instead of single steps, build targets without knowledge of the true labels, test shorter budgets, and measure the agent and the drone tools on a Jetson.

Acknowledgements

Experiments in this work were run and tracked with OpenResearch [25].

Data and code availability

The code, the saved tool answers and the result files are in the project repository, branch `agentic-ppo-v2` (to be made public on acceptance). ElecVQA-Bench and InsPLAD are public [22, 23]. The laya-vision weights are published under CC BY-NC-SA 4.0 [18].

Declaration of competing interest

The author declares no competing interests.

CRediT authorship contribution statement

Mahdi Khashan: Conceptualization, Methodology, Software, Investigation, Formal analysis, Writing – original draft, Writing – review & editing.

Declaration of generative AI use

During the preparation of this work the author used an AI assistant to write code, analyse results and draft text, and a language model to write the teacher reasons used as training data. The author reviewed and edited the content and takes full responsibility for the publication.

References

[1] R. Farahani, N. Hossein Motlagh, Z. Azimi, C. Timmerer, L. Carnevale, S. Tarkoma, S. Dustdar, From pixels to semantics: Edge AI for UAV-based critical infrastructure inspection, arXiv:2609.18448 (2026).
[2] R. Farahani, Z. Azimi, I. Murturi, A. Goknil, S. Sen, C. Timmerer, S. Dustdar, EVLM: Intent-driven edge vision language model for UAV-based power line inspection, in: 2026 IEEE International Conference on Edge Computing and Communications (EDGE), 2026. https://doi.org/10.1109/EDGE72783.2026.00014.
[3] Z. Azimi, R. Farahani, S. Dustdar, C. Timmerer, Cloud, edge, or split? Profiling onboard and split vision-language model deployment for drone AI, arXiv:2609.25415 (2026).
[4] Sabanovic, Maliakel, Brandic, INAR-VL: Input-aware routing for edge-cloud vision-language inference, arXiv:2605.18853 (2026).
[5] Zhou et al., Training small language models for transferable tool selection, arXiv:2610.31483 (2026).
[6] Z. Shao, et al., DeepSeekMath: Pushing the limits of mathematical reasoning in open language models, arXiv:2402.03300 (2024).
[7] A. Yang, et al., Qwen3 technical report, arXiv:2505.09388 (2025).
[8] T. Schick, et al., Toolformer: Language models can teach themselves to use tools, in: Advances in Neural Information Processing Systems (NeurIPS), 2023.
[9] S. Yao, et al., ReAct: Synergizing reasoning and acting in language models, in: International Conference on Learning Representations (ICLR), 2023.
[10] Y. Qin, et al., ToolLLM: Facilitating large language models to master 16000+ real-world APIs, in: International Conference on Learning Representations (ICLR), 2024.
[11] S. G. Patil, T. Zhang, X. Wang, J. E. Gonzalez, Gorilla: Large language model connected with massive APIs, arXiv:2305.15334 (2023).
[12] C.-Y. Hsieh, et al., Distilling step-by-step! Outperforming larger language models with less training data and smaller model sizes, in: Findings of the Association for Computational Linguistics (ACL), 2023.
[13] S. Robertson, H. Zaragoza, The probabilistic relevance framework: BM25 and beyond, Foundations and Trends in Information Retrieval 3 (4) (2009) 333–389.
[14] L. von Werra, et al., TRL: Transformer reinforcement learning, software (2020). https://github.com/huggingface/trl.

[15] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, arXiv:1707.06347 (2017).
[16] E. J. Hu, et al., LoRA: Low-rank adaptation of large language models, in: International Conference on Learning Representations (ICLR), 2022.
[17] A. Marafioti, et al., SmolVLM: Redefining small and efficient multimodal models, arXiv:2504.05299 (2025).
[18] Laya Vision, software, commit 9e1e241 (2026). `https://github.com/r33drichards/laya-vision`; weights `https://huggingface.co/thaitea/laya-vision`.
[19] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.
[20] M. Oquab, et al., DINOv2: Learning robust visual features without supervision, Transactions on Machine Learning Research (2024).
[21] A. Radford, et al., Learning transferable visual models from natural language supervision, in: International Conference on Machine Learning (ICML), 2021.
[22] ElecVQA-Bench, dataset and code repository (2026). `https://github.com/GhostDino/elecvqa-bench`.
[23] A. L. B. Vieira e Silva, et al., InsPLAD: A dataset and benchmark for power line asset inspection in UAV images, International Journal of Remote Sensing (2023). arXiv:2311.01619.
[24] J. van der Hooft, S. Petrangeli, T. Wauters, R. Huysegems, P. R. Alface, T. Bostoen, F. De Turck, HTTP/2-based adaptive streaming of HEVC video over 4G/LTE networks, IEEE Communications Letters 20 (11) (2016) 2177–2180.
[25] alphaXiv, OpenResearch: The local-first harness and workspace for research agents (2026). `https://github.com/alphaXiv/OpenResearch`.